
APACHE AIRFLOW 3.1.7



STAGE 1 / FOUNDATION

Foundation

Chạy Airflow local, DAG đầu tiên & TaskFlow API

Target:	Backend / Distributed Systems Engineer
Duration:	Ngày 1-2 (2-3 giờ/ngày)
Executor:	LocalExecutor (Docker)
Version:	Apache Airflow 3.1.7 (2026)

Mục lục

1. Mục tiêu học tập

2. Khởi động Airflow trên Docker

2.1. Docker Compose (LocalExecutor)

2.2. Lệnh khởi động

3. DAG đầu tiên — E-invoice validation

3.1. Cấu trúc DAG cơ bản

3.2. Các thành phần DAG

3.3. XCom và context

4. TaskFlow API (Modern approach)

4.1. Decorator @dag và @task

4.2. Tự động XCom với TaskFlow

5. Key Concepts Checklist

6. Bài tập thực hành

1. Mục tiêu học tập

Sau khi hoàn thành Stage 1, bạn sẽ đạt được các mục tiêu sau:

- **Hiểu rõ** khái niệm DAG, Operator, Task, Sensor ở mức hands-on
- **Chạy được** Airflow 3.1.7 trên Docker với LocalExecutor
- **Viết được** DAG đầu tiên sử dụng PythonOperator và BashOperator
- **Áp dụng được** TaskFlow API — phong cách viết DAG hiện đại của Airflow 3.x
- **Tránh được** các anti-pattern phổ biến khi mới bắt đầu

Stage 1 là nền tảng cho toàn bộ khóa học. Các kiến thức ở đây sẽ được mở rộng và đi sâu ở các Stage tiếp theo. Hãy dành thời gian thực hành kỹ trước khi chuyển sang Stage 2.

2. Khởi động Airflow trên Docker

Airflow cung nhiều cách cài đặt, nhưng với môi trường phát triển local, Docker Compose là lựa chọn tốt nhất. Nó đảm bảo tính nhất quán, dễ dàng xóa bỏ và tái tạo môi trường.

2.1. Docker Compose (LocalExecutor)

File `docker-compose.yml` dưới đây cấu hình Airflow 3.1.7 với LocalExecutor — phù hợp cho dev và small production. LocalExecutor chạy task trên cùng một node, sử dụng `fork()` để tạo process con cho mỗi task.

Tại sao LocalExecutor?

LocalExecutor đơn giản, không cần broker (Redis/RabbitMQ), phù hợp cho <50 DAGs. Chi phí thấp, dễ debug. Khi cần scale, ta sẽ chuyển sang CeleryExecutor ở Stage 4.

2. Khởi động Airflow trên Docker

```
# docker-compose.yml (Airflow 3.1.7 – LocalExecutor)
version: '3.8'
x-airflow-common:
  &airflow-common
  image: apache/airflow:3.1.7
  environment:
    &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
      postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__API__AUTH_BACKENDS: 'airflow.api.auth.backend.basic_auth'
    AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL: 120
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ./plugins:/opt/airflow/plugins
  depends_on:
    &airflow-common-depends-on
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
      start_period: 5s

  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports:
      - "8080:8080"
    depends_on:
      <<: *airflow-common-depends-on
      airflow-init:
        condition: service_completed_successfully

  airflow-scheduler:
    <<: *airflow-common
    command: scheduler
```

```

<<: *airflow-common-depends-on
  airflow-init:
    condition: service_completed_successfully

  airflow-init:
    <<: *airflow-common
    entrypoint: /bin/bash
    command:
      - -c
      - |
        mkdir -p /sources/logs /sources/dags /sources/plugins
        airflow db migrate
        airflow users create \
          --username admin \
          --password admin \
          --firstname Admin \
          --lastname User \
          --role Admin \
          --email admin@example.com
    depends_on:
      <<: *airflow-common-depends-on

  volumes:
    postgres-db-volume:

```

Bảng 1: Giải thích các service trong Docker Compose

Service	Vai trò	Port
postgres	Metadata database — lưu trạng thái DAG, task, XCom	5432 (internal)
airflow-webserver	Giao diện web UI và REST API	8080
airflow-scheduler	"Bộ não" của Airflow — lập lịch và điều phối task	Không expose
airflow-init	One-time init — tạo bảng DB và admin user	Không chạy liên tục

2.2. Lệnh khởi động

Chạy từng lệnh theo thứ tự sau:

```
# Bước 1: Khởi tạo database và admin user
docker compose up airflow-init

# Bước 2: Chạy các service chính ở chế độ detached
docker compose up -d

# Bước 3: Kiểm tra trạng thái
docker compose ps

# Bước 4: Truy cập UI tại http://localhost:8080
# Đăng nhập: admin / admin
```

Lưu ý quan trọng

Service `airflow-init` phải chạy thành công (`service_completed_successfully`) trước khi webserver và scheduler khởi động. Nếu gặp lỗi, kiểm tra logs bằng `docker compose logs airflow-init`.

3. DAG đầu tiên — E-invoice validation pipeline

Đây là DAG minh họa quy trình xử lý hóa đơn điện tử đơn giản. DAG này bao gồm 3 task: kiểm tra thư mục nguồn, validate XML, và thông báo hoàn thành.

3.1. Cấu trúc DAG cơ bản

Mỗi DAG định nghĩa bằng Python file trong thư mục `dags/`. File dưới đây đặt tại `dags/stage1_hello_dag.py`:

```

# dags/stage1_hello_dag.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

def validate_invoice_batch(**context):
    """Validate một batch hóa đơn – chạy trong worker process."""
    import random # lazy import – KHÔNG import ở top-level
    batch_id = context['ds'] # execution date (YYYY-MM-DD)
    result = {"batch_id": batch_id, "valid": random.choice([True, False])}
    print(f"Validated batch: {result}")
    return result

with DAG(
    dag_id='stage1_invoice_validation',
    default_args={
        'owner': 'tuanta',
        'depends_on_past': False,
        'email_on_failure': False,
        'retries': 1,
        'retry_delay': timedelta(minutes=5),
    },
    description='Stage 1: Basic invoice validation DAG',
    schedule=timedelta(hours=1), # Airflow 3.x: 'schedule' thay 'schedule_interval'
    start_date=datetime(2026, 6, 1),
    catchup=False, # QUAN TRỌNG: tránh backfill hàng loạt
    tags=['stage1', 'invoice'],
) as dag:

    check_source = BashOperator(
        task_id='check_source_dir',
        bash_command='echo "Checking /data/invoices/{{{{ ds }}}}"',
    )

    validate = PythonOperator(
        task_id='validate_xml',
        python_callable=validate_invoice_batch,
    )

    notify = BashOperator(
        task_id='notify_completion',
        bash_command='echo "Batch {{{ ds }}} completed"',
    )

    # Định nghĩa thứ tự thực thi
    check_source >> validate >> notify

```

3.2. Giải thích các thành phần

Bảng 2: Các thành phần chính trong DAG

Thành phần	Ý nghĩa	Ví dụ
DAG	Đồ thị có hướng, không chu trình (Directed Acyclic Graph)	Quy trình ETL tổng thể
Task	Đơn vị công việc — 1 node trong đồ thị	Validate XML, submit Spark job
Operator	Template để tạo Task — định nghĩa CÔNG VIỆC thực hiện	PythonOperator, BashOperator
Sensor	Operator đặc biệt — chờ đợi điều kiện xảy ra	FileSensor, ExternalTaskSensor
DagRun	1 lần chạy cụ thể của DAG tại 1 thời điểm	Run #124 cho 2026-06-02
TaskInstance	1 lần chạy cụ thể của Task trong 1 DagRun	validate_xml cho Run #124

Các tham số quan trọng

Bảng 3: Tham số DAG quan trọng

Tham số	Mục đích	Giá trị khuyến nghị
<code>schedule</code>	Tần suất chạy DAG (3.x unified thành 'schedule')	<code>timedelta(hours=1)</code> , '@daily'
<code>start_date</code>	Ngày bắt đầu — DAG sẽ chạy từ ngày này	Không quá khứ gần đây
<code>catchup</code>	Nếu True, sẽ chạy lại các khoảng thời gian đã qua	False cho production
<code>retries</code>	Số lần thử lại khi task thất bại	1-3
<code>retry_delay</code>	Thời gian chờ giữa các lần retry	<code>timedelta(minutes=5)</code>
<code>depends_on_past</code>	Task run này có phụ thuộc run trước không	False (trừ khi cần)

3.3. XCom và context

Trong hàm `validate_invoice_batch`, ta sử dụng `**context` để truy cập các biến môi trường. Đây là các biến sẵn có trong Airflow:

Bảng 4: Các biến context phổ biến trong Airflow

Biến	Kiểu	Mô tả
<code>ds</code>	str	Execution date dạng YYYY-MM-DD
<code>ds_nodash</code>	str	Execution date dạng YYYYMMDD
<code>ts</code>	str	Execution timestamp ISO format
<code>ti</code>	TaskInstance	Để push/pull XCom, truy cập metadata
<code>dag</code>	DAG	Đối tượng DAG hiện tại
<code>run_id</code>	str	ID của DagRun
<code>params</code>	dict	Tham số truyền vào DAG

Anti-pattern: Import nặng ở top-level

KHÔNG import pandas, spark, hay các thư viện nặng ở đầu file DAG. Điều này làm chậm quá trình parse DAG của scheduler. Hãy dùng **lazy import** bên trong hàm task.

4. TaskFlow API (Modern approach)

TaskFlow API là cách viết DAG hiện đại trong Airflow 3.x, sử dụng Python decorators `@dag` và `@task`. Nó tự động xử lý XCom, làm code gọn gàng và dễ đọc hơn.

Lưu ý quan trọng (Airflow 3.1.7): Airflow 3.x giới thiệu namespace `airflow.sdk` là giao diện ổn định cho DAG authoring. Tuy nhiên, các import từ `airflow.decorators` vẫn hoạt động để đảm bảo backward compatibility. Trong khóa học này, ta dùng cả 2 cách.

4.1. Decorator @dag và @task

File `dags/stage1_taskflow.py` — cùng chức năng nhưng viết bằng TaskFlow API:

```
# dags/stage1_taskflow.py
from airflow.decorators import dag, task
from datetime import datetime, timedelta

@dag(
    schedule=timedelta(hours=1),
    start_date=datetime(2026, 6, 1),
    catchup=False,
    tags=['stage1', 'taskflow'],
)
def stage1_taskflow_etl():
    """DAG sử dụng TaskFlow API – tự động XCom giữa các @task."""

    @task
    def extract() → dict:
        """Task 1: Trích xuất dữ liệu từ Kafka."""
        return {"records": 1000, "source": "kafka-topic-invoices"}

    @task
    def transform(data: dict) → dict:
        """Task 2: Biến đổi dữ liệu – nhận data từ task extract."""
        return {**data, "processed": True, "valid_count": 950}

    @task
    def load(data: dict) → None:
        """Task 3: Tải dữ liệu vào Cassandra."""
        print(f>Loading {data['valid_count']} records to Cassandra")

    # Định nghĩa luồng – tự động xcom giữa các task
    load(transform(extract()))

# Khởi tạo DAG
dag_instance = stage1_taskflow_etl()
```

4.2. Tự động XCom với TaskFlow

Khi dùng TaskFlow API, việc truyền dữ liệu giữa các task trở nên tự nhiên:

```
# Traditional API — phải xcom_push/xcom_pull thủ công
def push_func(ti, **kwargs):
    ti.xcom_push(key='my_key', value='my_value')

def pull_func(ti, **kwargs):
    value = ti.xcom_pull(task_ids='push_task', key='my_key')

# TaskFlow API — tự động qua return value và tham số
@task
def task_a() → str:
    return "result_a" # Tự động xcom_push

@task
def task_b(value: str) → str: # Tự động xcom_pull từ task_a
    return f"received: {value}"
```

Bảng 5: So sánh Traditional API và TaskFlow API

Tiêu chí	Traditional API	TaskFlow API
Cú pháp	Verbose — phải định nghĩa Operator, task_id	Ngắn gọn — decorators
XCom	Thủ công qua ti.xcom_push/pull	Tự động qua return/param
Data typing	Không có	Có type hints
Phù hợp	Custom operator, phức tạp	ETL pipeline, ML pipeline

5. Key Concepts Checklist

Trước khi chuyển sang Stage 2, hãy đảm bảo bạn nắm vững các khái niệm sau:

- **DAG** là đồ thị có hướng, không chu trình (Directed Acyclic Graph). Mỗi node là 1 task, mỗi cạnh là dependency.
- **schedule vs schedule_interval**: Airflow 3.x đã thống nhất thành `schedule`. Cú pháp cũ `schedule_interval` vẫn hoạt động nhưng deprecated.
- **catchup=False** là bắt buộc cho production. Nếu đặt True, Airflow sẽ chạy lại tất cả các khoảng thời gian từ `start_date` đến hiện tại — có thể tạo ra hàng trăm/thậm chí hàng nghìn run cùng lúc.
- **retries và retry_delay** nên có trên mọi task. Không có retry = task fail 1 lần là pipeline fail.
- **TaskFlow API** tự động xử lý XCom giữa các task qua return values và function parameters. Không cần xcom_push/pull thủ công.

- **Lazy import** — luôn import thư viện nặng bên trong hàm task, không ở top-level file.
- **Jinja templating** — sử dụng `{{ ds }}`, `{{ macros }}` để truy cập biến động trong các tham số Operator.

6. Bài tập thực hành

Các bài tập dưới đây giúp bạn củng cố kiến thức Stage 1:

Bảng 6: Bài tập Stage 1

STT	Bài tập	Mục tiêu
1	Chạy <code>docker compose up</code> và truy cập UI ở localhost:8080	Thành thạo Docker workflow
2	Viết DAG với 3 tasks: <code>extract</code> → <code>transform</code> → <code>load</code> (BashOperator)	Hiểu dependency và operator
3	Convert DAG sang TaskFlow API (<code>@dag</code> , <code>@task</code>)	Thành thạo TaskFlow
4	Chạy <code>airflow dags test stage1_invoice_validation 2026-06-01</code>	Biết cách test DAG qua CLI
5	Thêm <code>retries=2</code> và <code>retry_delay=timedelta(minutes=1)</code>	Hiểu cơ chế retry
6	Thử <code>catchup=True</code> với <code>start_date</code> cũ → quan sát backfill → đổi về False	Thấy tác hại của catchup

Mẹo kiểm tra

Sau mỗi bài tập, vào UI **Browse** → **DAG Runs** và **Browse** → **Task Instances** để quan sát kết quả. Đọc logs bằng `docker compose logs airflow-scheduler` khi gặp lỗi.